

[← Go Back](#)

Hardware

MKR WAN 1310

Tutorials

[LoRa® LED Control with MKR WAN 1310](#)[LoRa®-Based Messaging with MKR WAN 1310](#)[Send Data Using LoRa® Technology with MKR WAN 1310](#)[LoRa® Sensor Data with MKR WAN 1310](#)[LoRa® Farming with MKR WAN 1310](#)[LoRa® Regional Parameters in Arduino® MKR WAN 1310](#)[MKRWAN Examples](#)[Connecting MKR WAN 1310 to The Things Network \(TTN\)](#)[MKR WAN 1310 with MKR GPS Shield](#)[Home](#) / [Hardware](#) / [MKR WAN 1310](#) / [LoRa® Farming with MKR WAN 1310](#)

LoRa® Farming with MKR WAN 1310

Let's collect sensor data with MKR WAN 1300 or 1310 and combine it with weather forecast in order to optimize watering in a wide area!

Author · [FabLab_CastelfrancoVeneto](#), Last revision · [02/28/2025](#)

Components and Supplies

[Arduino MKR WAN 1300](#)[ARDUINO MKR CONNECTOR CARRIER \(GROVE COMPATIBLE\)](#)[USB power supply](#)[Seeed Grove - Relay](#)[Seeed Grove - Temperature&Humidity Sensor](#)[Seeed Grove - Light Sensor](#)[Seeed Grove Moisture Sensor](#)

Apps and Online Services

[Arduino Cloud Editor](#)[The Things Network](#)[Node-RED](#)[OpenWeatherMap](#)

About This Project

If you need to control a watering system (or lighting system or other generic actuators) in a smart way and collect sensor data over a large area where WiFi coverage is optional, you have to switch to other ways to communicate.

LoRa® technology is an excellent choice due to its low power consumption, long range and cost effective hardware. Most importantly, LoRa devices can operate within public or private networks.

ON THIS PAGE

Components and Supplies

[Apps and Online Services](#)[About This Project](#)[Electronics Setup](#)[TheThingNetwork Setup](#)[The Arduino MKR 1300 / MKR 1310 Code](#)[Node-RED Setup](#)[Node-RED Integration with TTN and Sensor Data Visualization](#)[Node-RED and Scheduler + OpenWeatherMap \(for Weather Forecast\)](#)[Complete Sketch](#)[Trademark Acknowledgments](#)

In this project we'll use the most known global LoRa®-based network: **The Things Network**.

We'll then integrate everything with **Node-RED**, so it will be easy to plot the received data and create rules to send back commands!

As for initial setup, please follow our previous guide "MKR WAN 1310 meets TheThingsNetwork!"

Steps will be:

- Electronics setup

- The Thing Network setup

- The Arduino MKR WAN 1300 and MKR WAN 1310 code

- Node-RED setup

- Node-RED integration with TTN and sensor data visualization

- Node-RED and scheduler + OpenWeatherMap (for weather forecast)

Let's start!

Electronics Setup

Here it's quite easy, thanks to the MKR Connector Carrier and the Seeed Grove modules!

We'll use here, to monitor the environment:

- A light sensor module (connected to A0 header on MKR Connector Carrier)

- A moisture sensor module (connected to A3)

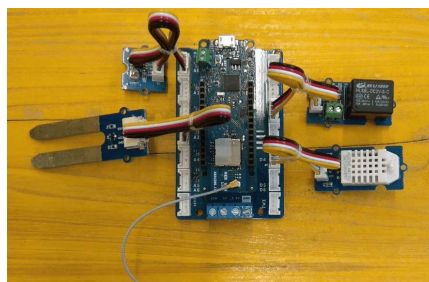
- A temperature and humidity sensor module (connected to D4)

and to activate the watering system

- A relay module (connected to D2)

Then just fit the Arduino MKR WAN 1300 or MKR WAN 1310 the on the top of the Connector Carrier, and don't forget to attach its antenna as well.

Final setup should be like this:



The final hardware setup.

The moisture sensor will be later fitted in the soil, and the relay connected to a proper power supply and to a valve or a water pump.

This is a simple setup, but you can improve accuracy on watering i.e. using a water flow meter.

Of course you can reach the same setup using a breadboard and spare components!

TheThingNetwork Setup

If you have followed our previous guide

`"MKR WAN 1310 meets TheThingsNetwork!"`, then you are done.

The Arduino MKR 1300 / MKR 1310 Code

Only few things to say about the code (that you'll find below). This is based on the

`LoraSendAndReceive` example available in the MKRWAN library.

There are 2 additional libraries imported:

The DHT library is used to read the DHT22 sensor

The *ArduinoJSON* library is used to parse the *_downlink* message from TTN

The reading of the sensors is done at the interval specified with the *interval* constant.

After reading the sensors, we prepare a JSON string with all the data collected for the TTN *uplink*.

If there's a *downlink* available, it will contain a command for change status to our relay, that will be:

Turned on if we need to water our plant(s)

Turned off if it's time to stop watering!

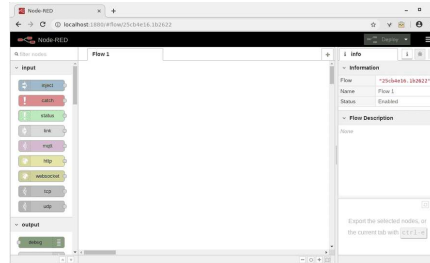
The *downlink* message will be send only if there's a status change.

Node-RED Setup

Node-RED is a flow-based programming tool born for an easy implementation and deploy of IoT apps.

It's available for every OS, and well documented.
After **installing it**, just **run it** and point your browser to:

<http://localhost:1880>



You should see a page like this one.

Now you'll have to install additional palettes (kinda plugins) in order to:

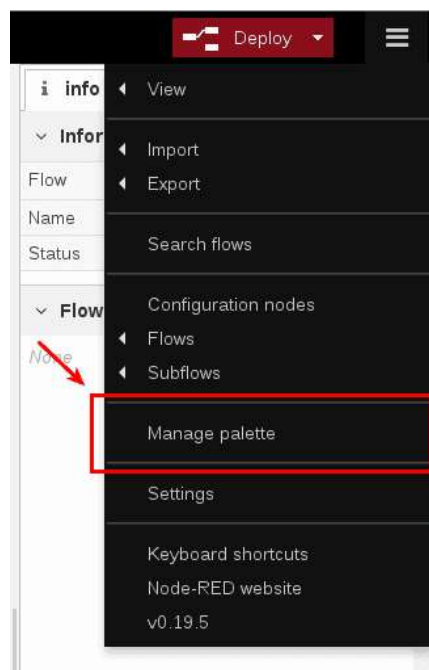
Connect to The Things Network (TTN) to retrieve *uplinks* from the MKR WAN 1300 and MKR WAN 1310 and send back *downlink* messages.

Create a dashboard for the sensor data visualization

Define a calendar for the watering

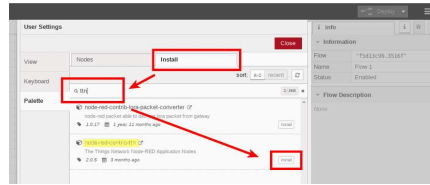
Verify the weather forecast for a close rain (in that case watering won't make sense)

To install a new palette, just click on the burger icon in the top right corner, and choose "Manage Palette"



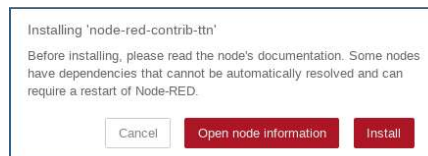
Choose manage palette.

Click on the *install* button and then fill the search field with "ttn"; it will filter the available palettes. Let's install the *node-red-contrib-ttn* one by pressing the *install* button in the filtered result.



Click on install and search for "ttn".

You have also to confirm the installation



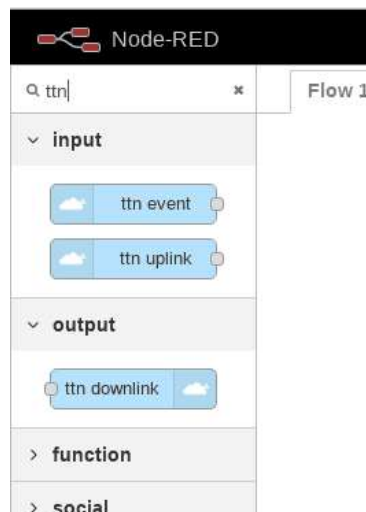
Confirm the installation.

Now do the same for the following palettes:

node-red-node-openweathermap
node-red-contrib-light-scheduler
node-red-node-rbe
node-red-dashboard
node-red-contrib-startup-trigger

These new palettes will add new nodes on the column on the left.

You can check for new nodes also using the filter on the top of that column:

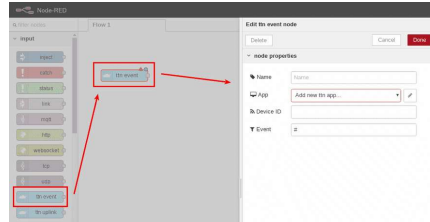


ttn Nodes.

Node-RED Integration with TTN and Sensor Data Visualization

In order to use the TTN and OpenWeatherMap nodes, we have to do a first setup.

From the nodes on the left column, drag a "ttn uplink" node into the *flow* (kinda program) in the main area.

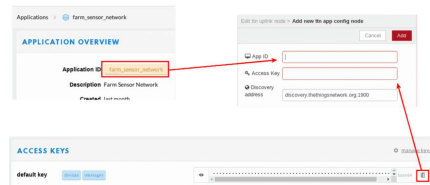


Drag the node into the flow.

In the right area, we can configure it. For first set the *App* adding a new ttn app. Click on the pencil icon and fill:

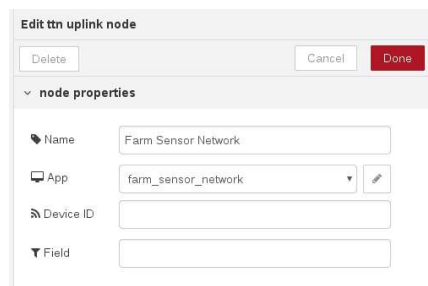
the App ID field with the name chosen on TTN for your App, and

the AccessKey by copy-paste the Access Key from the same App page



Let the Discovery address untouched, and confirm by pressing the Add button.

Now set a name for this node. Let the *Device ID* and the *Field* fields empty (in this way this node will receive the *uplinks* from all the devices registered in that app!) and press *Done*.



Name the node.

This first node of the flow will receive from the ttn app every *uplink* (in other words: all the messages sent by the Arduino MKR WAN 1300 or the MKR WAN 1310).

Start adding other nodes to the flow:

The second one will extract the decoded message. This node is a *function* node, and it's used to process an input i.e. in order to extract a part of the message. Here the programming language is *javascript*



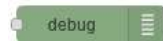
Function node.

The third one will transform the input into a JSON object; it's indeed a *JSON* node



Json node.

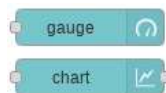
At this point we can use the *debug* node to show in the debug area (left column, bug icon) the output



Debug node.

In the same way we process that JSON in order to extract the light, temperature, humidity and moisture values thanks to other few *function* nodes

In the end, with these data, we can create a dashboard: we can add a *gauge* widget and a *chart* widget for each sensors' value



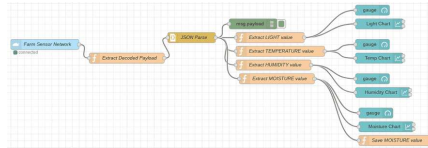
Gauge and chart widgets.

Last but not least: we have also to save somewhere the value of the moisture; it will be used later on. For this we use another *function* node, in which will save in the *flow* that value. You can find more info on storing data on Node-RED [at this link](#).

When you use the first *dashboard* node into a flow,

and you can find more info in the [node-red-dashboard](#) node page, *Layout* section.

Connect then in the right order the nodes: you should have now something like this:

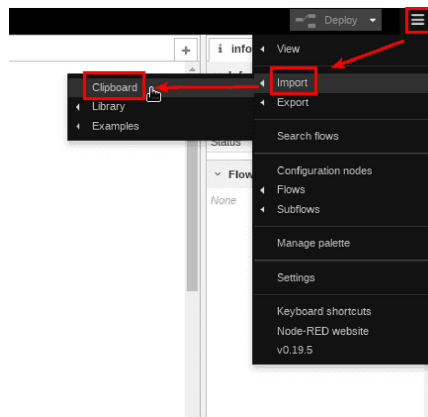


The flow.

Here you can find this flow ready to be imported in your Node-RED!

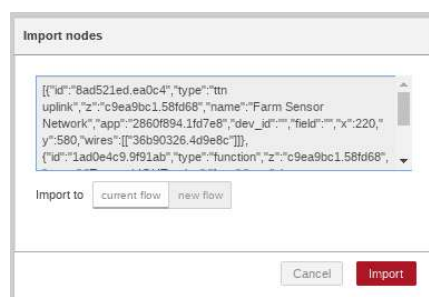
```
1 [{"id":"8ad521ed.ea0c4","type":"ttn u
```

Just select and copy it; then click on the burger icon on the top left corner, and choose *Import* and then *Clipboard*



Import from clipboard.

Now paste the flow code, choose to import in the *current flow*, and press *Import*



Import to current flow and press import

Remember to fix the TTN node configuration!

Once done, is time to run the flow. Press the *Deploy* button on the top right corner (this is how a flow is validated and executed).

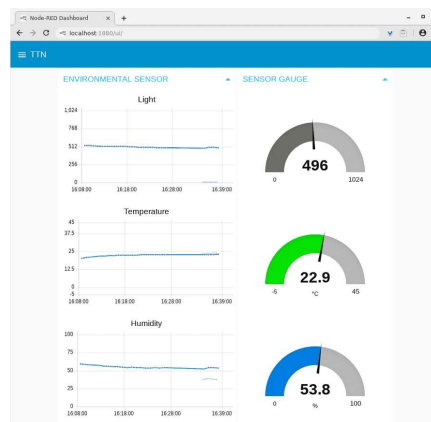


Press the deploy button.

Now you can also open the dashboard; point your browser to:

<http://localhost:1880/ui>

You should see something like this (widgets will be empty until a few data will arrive)



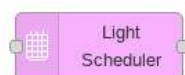
Node-RED dashboard.

If everything went fine until now, we have finished the first part of the project; the monitoring part.

Node-RED and Scheduler + OpenWeatherMap (for Weather Forecast)

Now we need to schedule the watering, and avoid water waste due to incoming rain event or soil already wet enough.

Let's start using the *light scheduler* node, that is good to schedule also stuffs other than light:



Editing this node (double click on it, once dragged into the flow) allows us to define a weekly scheduling, to set Latitude and Longitude of our "farm" in order to calculate sunshine / sunrise (in the Globals field), and to set the output method: here set

For *On Payload* an output *Boolean True*,
For *Off Payload* an output *Boolean False*
For output "*when state change + startup*"

Edit Light Scheduler node

Delete Cancel Done

node properties

Name Scheduler

Globals FabLabCfv

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
4:00							
5:00							
6:00							
7:00							
8:00							
9:00							
10:00							

Note: Marked schedule = "On Payload"

Output

Topic Topic

On Payload true

Off Payload false

Output when state changes + startup

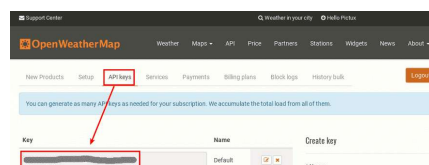
Node properties.

Another node to use is the *openweathermap* one.



Openweathermap node.

In order to use this node you have to obtain a API KEY from OpenWeatherMap. It's very easy: just register on [their site](#) for a Free Plan and then copy your API KEY!



Copy your API key.

The Free Plan allows you to do up to 60 calls for minute (that is a lot).

Then you have to edit the node property filling the data about your location, by coordinates or by city name.

Edit the node properties with your location.

The output of this node will be the forecast for the next 5 days, divided in 3 hours forecast slots.

By using a *function* node add a few rows of javascript code on it, we'll keep only the data about precipitation volume of the first 3 slots (that means a total of 9 hours forecast). If the total amount will be over a certain threshold, of course we won't water!

Of course another node to use is the *ttn downlink*



ttn downlink node.

It will be used to send back the relay status update to the Arduino MKR WAN 1300 or the MKR WAN 1310.

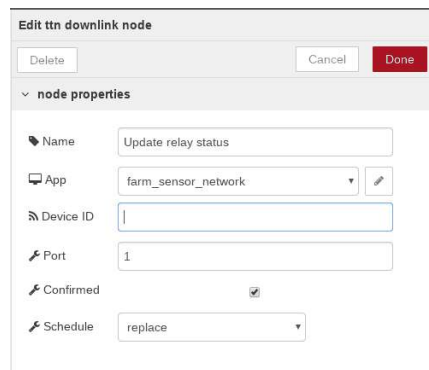
Once imported into the flow, edit its property:

set a proper (arbitrary) *Name*

choose from the dropdown field the *App* you have already configured for the *ttn uplink* node

don't specify the *Device ID* that will be the recipient of the message (in this way we can use this node for send message to more than one device! Later on we'll see how)

Select *replace* value for *Schedule*



Node properties settings.

After these, we need other nodes in order to make everything work:

rbe node (Report By Exception), it is used to generate an output only if there's a change in the output from the Light Scheduler (from ON to OFF and vice versa)



rbe Node.

A few *function* nodes, used to check for the forecast and for build the *downlink* message (in which we'll specify also the device id)



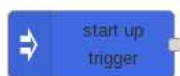
Function node.

A few *switch* nodes in order to define rules based on different input (i.e. the moisture value, or the amount of forecast rain, etc)



Switch node.

Last node we need is the *start-up-trigger* node, that will be used start the *scheduler* at every boot



Start-up trigger node.

The *scheduler* is the main in command. It will generate an output that could be true or false.

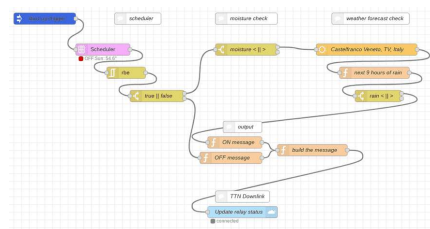
If the output is **false**, send directly the *downlink* message to the device in order to turn OFF the relay.

If the output is **true**, there will be for first a check on the last value registered for the moisture sensor. If it's over a certain threshold, we'll not water this time.

If the moisture value is under that threshold, then we check the weather forecast; if there's a certain amount of forecast rain in the next 9 hours, again: no need to water.

Otherwise, send the *downlink* message to the device in order to turn ON the relay

The flow will be something like this:



Current flow.

Here you can find this flow ready to be imported in your Node-RED! Remember to update TTN and OpenWeatherMap nodes configuration.

And of course, you can change the values used as threshold on the *switch* nodes.

```
1 [{"id":"197f29f7.4643d6","type":"ligh
```

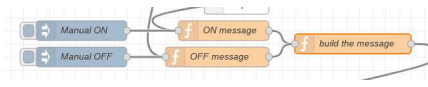
Now you have a complete setup: you can start doing precision farming.

Just start testing the *downlink* message: for this you can add to your flow a couple of *inject* nodes (usually used to trigger manually the execution of a flow)



Inject node.

You don't need to change anything on them but the name. and link them to the *function* nodes used to



Click on the square on the left of the node "Manual ON".

Just click on the square on the left of the node *Manual On*: you'll generate an event that will build and send to TTN the *downlink* message for turn on the relay.

Remember that the *downlink* message is sent immediately by Node-Red to TTN (in fact you can see it in queue on the [TTN Console](#)), but it is sent back to the Arduino MKR 1300 only after an *uplink* message!

▼ 18:16:34 1 scheduled confirmed payload: 7B 22 73 74 61 74 75 73 22 3A 31 7D

Payload hex.

(payload is converted in hex base; you can use [this tool](#) to verify the content)

Now open the Monitor (or Serial Monitor if you are working on the IDE) and you'll see in a while something like this:

```

1 Sending: {"light":364,"temp":25.70,"h
2 Message sent correctly!
3 Received: {"status":1}

```

After this first test, start playing with scheduler and with the moisture sensor (keep it dry for first, then try putting it into a wet sponge).

After you have tested indoor the flows and the circuit, it's time to apply it in the real world!

You can of course add more Arduino MKR WAN 1300 or MKR WAN 1310 devices in the TTN App, and handle them in Node-RED: you need to adapt the flows but as you have already seen on this project, both the hardware and software tools are very easy to use.

Complete Sketch

```
1  /*
2    Mirco Piccin, 04/2019
3    Based on Lora Send And Receive
4    This sketch used in
    combination with TTN and Node-
    Red will help you on
    implementing a Precison Farming
    scenario!
5    This example code is in the
    public domain.
6  */
7
8  #include <MKRWAN.h>
9  #include <Arduino_JSON.h>
10 #include "DHT.h"
11
12 LoRaModem modem;
13
14 String appEui = SECRET_APPEUI;
15 String appKey = SECRET_APPKEY;
16
17 //define DHT pin, model and
    create a DHT object
18 #define DHTPIN 4
19 #define DHTTYPE DHT22
20 DHT dht(DHTPIN, DHTTYPE);
```

Trademark Acknowledgments

LoRa® is a registered trademark of Semtech Corporation.

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

- [Help Center](#)
- [Ask the Arduino Forum](#)
- [Discover Arduino](#)
- [Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

